

ROADRESQ

Testing & QA Plan

Version 1.0 • Production-Oriented Quality Baseline

Scope: Web • Mobile • API • Database • Realtime • Payments • Security • Infrastructure

Quality objective: Safe, reliable and observable customer/provider service lifecycle

BUILD → TEST → VALIDATE → RELEASE → MONITOR → IMPROVE

Document Control

Field	Value
Document	Testing & QA Plan
Product	RoadResQ
Version	1.0
Status	Production QA baseline
Audience	QA, developers, product, DevOps, security and operations
Scope	Customer app/web, provider app/web, admin, APIs, DB, realtime, integrations and infrastructure

1. Purpose

This document defines the quality strategy, test levels, environments, test data, automation approach, release gates, defect management and production-readiness criteria for RoadResQ. The goal is to validate both functional correctness and real-world reliability across the complete assistance lifecycle.

1.1 Quality Goals

- Ensure customers can request and complete assistance safely.
- Ensure providers receive only eligible requests and cannot create conflicting assignments.
- Ensure booking state transitions remain consistent and auditable.
- Protect payments, personal data, documents and realtime location.
- Validate graceful behavior under network loss, provider rejection, payment failure and external-service outages.
- Detect regressions automatically before production.
- Make production behavior observable and recoverable.

2. Testing Scope

Area	In scope
Customer experience	Authentication, vehicles, request flow, provider discovery, tracking, estimate approval, payment, history and reviews
Provider experience	Registration, verification, services, availability, requests, acceptance, navigation/status, inspection, estimates, completion and earnings
Admin	Users, providers, verification, bookings, payments, support, disputes, audit and analytics
Backend/API	REST endpoints, validation, authorization, state machine, transactions, idempotency and errors
Realtime	WebSocket authentication, booking events, location updates, reconnect/recovery
Database	Constraints, migrations, queries, transactions, geospatial behavior and backup/restore
Integrations	Maps, payment gateway, OTP/SMS, push/email and object storage
Security	Authentication, RBAC, object authorization, uploads, rate limits, secrets and common vulnerabilities
Infrastructure	Containers, CI/CD, health checks, scaling, logs, metrics and failure recovery

3. Quality Strategy

RoadResQ should use a risk-based testing pyramid: many fast unit tests, a strong integration layer, targeted API/contract tests, realistic end-to-end flows and specialized performance/security testing. Critical business and financial paths receive deeper coverage than low-risk presentation-only features.

3.1 Test Pyramid

UNIT → COMPONENT → INTEGRATION → API/CONTRACT → E2E → PERFORMANCE/SECURITY/RESILIENCE

- Unit tests catch business-rule regressions early.
- Integration tests validate database and external adapter boundaries.
- API/contract tests ensure frontend/backend compatibility.
- E2E tests validate real customer-provider journeys.
- Performance/security/resilience tests validate production characteristics.

4. Test Levels

Level	Purpose	Examples
Unit	Smallest business logic	State transitions, price calculations, matching scores, validators
Component	Module behavior	Booking service, dispatch module, notification service
Integration	Real dependencies/boundaries	FastAPI + PostgreSQL, Redis, storage adapter, gateway adapter
Contract	API compatibility	OpenAPI request/response schemas and status codes
E2E	Complete workflows	Customer request → provider acceptance → completion → payment
Regression	Prevent existing functionality from breaking	Automated critical-path suite each release
Performance	Capacity/latency	Concurrent requests, dispatch, WebSockets, admin queries
Security	Threat prevention	Authorization, injection, uploads, rate limits, secrets
Resilience	Failure recovery	DB transient failure, Redis outage, provider API outage, reconnect

5. Test Environments

Environment	Purpose	Data / integrations
Development	Developer tests and local debugging	Synthetic data; mocked/test integrations
CI	Automated PR validation	Ephemeral/sandbox database and services
Staging	Production-like validation	Sanitized/synthetic data; sandbox payment/maps/notifications
Production	Real users	Real integrations; tightly controlled tests only

Production must not be used as a general-purpose QA environment. Any production verification must use controlled, non-destructive smoke checks.

6. Test Data Strategy

- Use synthetic customers, providers, vehicles, bookings and payment records for automated testing.
- Maintain deterministic seed data for CI and staging.

- Create test fixtures for every booking state.
- Include valid, invalid, boundary and malicious inputs.
- Never place real passwords, OTPs, payment secrets or unnecessary personal data in test repositories.
- Use masked/sanitized data if production-like records are ever required.
- Test Indian-context values such as INR amounts, UPI flows, local phone formats and geospatial coordinates.

7. Functional Test Matrix — Customer

ID	Scenario	Expected result	Priority
CUS-001	Register/login	Valid account/session created	P0
CUS-002	Invalid OTP	Verification rejected; rate controls apply	P0
CUS-003	Add vehicle	Vehicle saved and linked to owner	P0
CUS-004	Access another user's vehicle	403/404; no data leakage	P0
CUS-005	Create assistance request	Booking created with correct state/location	P0
CUS-006	Duplicate active request	Duplicate prevented or explicit policy applied	P0
CUS-007	No provider found	Clear no-match state + retry/alternative	P0
CUS-008	Provider accepts	Customer sees authoritative assignment	P0
CUS-009	Track provider	Location/status updates displayed	P0
CUS-010	GPS unavailable	Manual location fallback works	P1
CUS-011	Approve estimate	Approval persisted and work can proceed	P0
CUS-012	Reject estimate	Correct pending/requote/cancel state	P0
CUS-013	Payment success	Server-confirmed payment state displayed	P0
CUS-014	Payment timeout	Payment remains pending/unknown until reconciled	P0
CUS-015	Review completed job	Eligible review saved once	P1

8. Functional Test Matrix — Provider

ID	Scenario	Expected result	Priority
PRO-001	Register provider	Profile created in pending verification	P0
PRO-002	Upload invalid document	Rejected by validation/security controls	P0
PRO-003	Admin verifies provider	Provider becomes eligible according to policy	P0
PRO-004	Go online/offline	Availability changes correctly	P0
PRO-005	Receive eligible request	Request contains required context	P0
PRO-006	Two providers accept same request	Exactly one assignment succeeds	P0
PRO-007	Reject request	Request returns to dispatch pool if eligible	P0
PRO-008	Mark on-the-way/arrived	Valid state transitions recorded	P0
PRO-009	Create estimate	Line items/totals stored correctly	P0
PRO-010	Perform unapproved work	Server prevents controlled transition	P0
PRO-011	Complete job	Completion recorded; invoice workflow triggered	P0
PRO-012	View unrelated booking	Access denied	P0

9. Functional Test Matrix — Admin & Support

ID	Scenario	Expected result	Priority
ADM-001	Admin login	Privileged session established	P0
ADM-002	Support accesses admin-only function	Denied	P0
ADM-003	Verify provider	Decision recorded with audit event	P0
ADM-004	Suspend provider	Provider becomes unavailable as defined	P0
ADM-005	View active bookings	Operational data visible according to role	P0
ADM-006	Resolve dispute	Resolution state and audit trail recorded	P0
ADM-007	Change sensitive account status	Permission + audit event required	P0
ADM-008	Access audit logs	Authorized admin can investigate events	P1

10. Booking State Machine Testing

Authoritative lifecycle:

REQUESTED → SEARCHING → PROVIDER_ASSIGNED → ACCEPTED → ON_THE_WAY → ARRIVED → INSPECTION → ESTIMATE_PENDING → APPROVED → IN_PROGRESS → COMPLETED → INVOICE_GENERATED → PAYMENT_COMPLETED → CLOSED

Exception states: CANCELLED, EXPIRED, DISPUTED.

Test class	Examples
Valid transitions	Every permitted state transition succeeds with correct actor and prerequisites.
Invalid transitions	Client attempts arbitrary state changes; server returns conflict/validation error.
Role checks	Customer/provider/support/admin each attempt allowed and disallowed transitions.
Concurrency	Multiple provider accepts, duplicate completion, concurrent estimate approval.
Recovery	Server crash/retry during transition does not create inconsistent state.
Audit	Every important transition records actor, time, previous/new state and reason where applicable.

11. API & Contract Testing

- Validate every documented endpoint against its OpenAPI schema.
- Verify authentication and authorization for each protected endpoint.
- Validate request field types, required fields, boundaries and malformed payloads.
- Verify status codes and structured error responses.
- Test pagination, filtering and sorting allow-lists.
- Test idempotency for bookings, payment operations and critical writes.
- Verify object-level authorization with cross-user IDs.
- Verify consistent request IDs and safe error messages.

12. Database Testing

- Migration up/down or safe rollback strategy where applicable.
- Foreign-key and uniqueness constraints.

- Transaction atomicity for booking assignment and financial operations.
- PostGIS proximity queries with known coordinates.
- Index/query-plan checks for high-volume access patterns.
- Concurrent update tests.
- Backup creation and restore verification.
- Retention/deletion behavior for sensitive data.

13. Realtime & WebSocket Testing

Scenario	Expected result
Authenticated connect	Valid user connects successfully.
Unauthorized booking subscription	Connection/subscription denied.
Status event	Correct authorized clients receive event.
Location event	Only eligible customer/provider/admin channels receive location.
Reconnect	Client reconnects and reconciles authoritative REST state.
Stale location	UI/API handles stale data without false precision.
Message flood	Rate/message limits prevent resource exhaustion.
Multiple instances	Events are delivered consistently when horizontally scaled.

14. Payment Testing

- Successful UPI/payment flow.
- Payment failure and cancellation.
- Delayed gateway response.
- Duplicate payment request with same idempotency key.
- Same payment webhook delivered multiple times.
- Webhook signature invalid.
- Webhook arrives out of expected order.
- Client claims payment success while gateway says pending/failure.
- Invoice amount mismatch detection.
- Refund/reversal behavior when implemented.
- Provider payout/reconciliation consistency.

No test should use live payment credentials unless explicitly authorized and isolated. Production payment verification must use safe operational controls.

15. Security QA

Test area	Required checks
Authentication	Brute-force resistance, OTP expiry/attempts, session expiry/revocation
Authorization	RBAC + object-level access for every sensitive resource
Injection	SQL/NoSQL/command/template injection defenses where applicable
XSS	Reviews/chat/profile/media metadata safely rendered

Test area	Required checks
CSRF	Protected when cookie-based auth requires it
Uploads	Type/size validation, malicious files, private access
Rate limits	OTP, login, booking, location and other abuse-sensitive APIs
Secrets	Secret scanning; no credentials in source/logs/client bundles
WebSockets	Auth, channel authorization, message validation and limits
Payments	Webhook signature, replay/idempotency, reconciliation
Dependencies	Automated vulnerability scanning and remediation

16. Performance & Load Testing

Performance tests should model realistic traffic rather than only isolated endpoint benchmarks.

Scenario	Load focus
Assistance requests	Concurrent request creation and validation
Dispatch	Provider discovery, filtering, ranking and acceptance races
Realtime tracking	Concurrent WebSocket connections and location events
Bookings	Status updates and history reads/writes
Payments	Concurrent payment creation/webhooks
Admin	Large booking/provider/payment queries
Media	Upload and retrieval behavior under realistic file sizes

Performance gates should be based on measured latency, throughput, error rate, resource utilization and saturation points. Final SLO thresholds should be established from load-test results.

17. Resilience & Recovery Testing

- API instance restart during active booking.
- WebSocket disconnect/reconnect.
- Redis temporary outage.
- Database transient failure.
- Worker crash and retry.
- Payment provider outage.
- Maps/ETA provider outage.
- Notification provider outage.
- Object-storage temporary failure.
- Network latency/packet loss between client and API.
- Duplicate asynchronous job execution.

Expected principle: recover from persisted authoritative state and avoid duplicate irreversible effects.

18. Accessibility & UX QA

- Keyboard navigation for web admin/customer interfaces where applicable.
- Visible focus states.

- Form labels and useful validation messages.
- Readable contrast and typography.
- Screen-reader semantics for important controls.
- Responsive behavior on supported mobile/desktop sizes.
- Clear loading, empty, offline, permission and error states.
- Critical emergency request flow remains understandable under stress.

19. Browser, Device & Compatibility Matrix

Target	Baseline testing
Desktop web	Latest supported Chrome/Edge/Firefox; defined support window
Mobile web	Current major Android/iOS browsers
Android app	Supported OS range defined before release; representative low/mid/high devices
iOS app	Supported iOS range defined before release; representative devices
Network	Wi-Fi, 4G/5G, weak/intermittent connectivity
Location	Permission granted, denied, stale and unavailable GPS states

20. Defect Management

Severity	Definition	Release rule
P0 Critical	Security breach, data loss, payment integrity failure, core service unavailable	Must fix before release
P1 High	Core customer/provider journey blocked or severe degradation	Must fix before production
P2 Medium	Important feature defect with workaround	Fix or explicitly accept
P3 Low	Minor UI/content/non-critical issue	May defer with backlog decision

Every defect should include reproduction steps, environment, expected vs actual result, evidence, severity, affected version and linked requirement/test case.

21. CI/CD Quality Gates

Pull Request → Lint/Type Check → Unit Tests → Integration Tests → Security/Dependency Scan → Build → Contract Tests → Staging Deployment → E2E → Approval → Production

- PR cannot merge when mandatory checks fail.
- Critical security findings block release unless explicitly risk-accepted.
- Database migrations are validated before deployment.
- Production deployment requires successful staging validation.
- Smoke tests run immediately after production deployment.

22. Regression Strategy

- **Smoke suite:** login, health, create booking, provider acceptance, status update and basic payment state.
- **Critical regression:** full customer-provider lifecycle and authorization boundaries.
- **Extended regression:** notifications, support, history, admin workflows and edge cases.
- **Release-specific:** tests for the feature changed plus affected integrations.

23. Production Smoke Tests

Check	Expected result
API health	Healthy status without exposing secrets.
Authentication	Safe test account can authenticate.
Customer data	Profile/vehicle read works.
Booking path	Controlled non-destructive test path works where operationally approved.
Realtime	WebSocket connectivity and safe event path verified.
Payment	Gateway connectivity/reconciliation health verified without unsafe live charges.
Notifications	Delivery infrastructure health verified.
Observability	Logs/metrics/alerts receiving expected telemetry.

24. Release Readiness Checklist

- All P0/P1 defects closed.
- Required automated test suites pass.
- Critical E2E journeys pass in staging.
- Authorization tests pass.
- Payment/idempotency/webhook tests pass.
- Realtime reconnect and location privacy tests pass.
- Security scan has no unresolved critical findings.
- Load/performance thresholds are met or formally accepted.
- Database migrations validated.
- Backup/restore readiness confirmed.
- Monitoring and alerting verified.
- Rollback plan tested/available.
- Release notes and known limitations documented.

25. QA Metrics

Metric	Purpose
Pass rate	Overall automated/manual test health
Defect escape rate	Defects reaching later environments/production
P0/P1 count	Release risk indicator
Mean time to resolution	Defect response efficiency
Flaky test rate	Automation reliability
Coverage	Identify untested code/rules; not a standalone quality score
E2E critical-path pass rate	Confidence in customer/provider journey
Performance error rate	Production-like capacity health
Security finding age	Track unresolved risk

26. Test Ownership

Role	Responsibility
Developer	Unit/component tests, defect fixes, technical test support
QA	Test planning, E2E/regression, defect validation, release recommendation
Security	Threat-based security testing and vulnerability review
DevOps	Environment, CI/CD, observability, resilience and deployment testing
Product	Acceptance criteria and business behavior validation
Operations	Real-world workflow and support/dispute acceptance

27. Definition of Done — Feature

- Acceptance criteria implemented and reviewed.
- Unit/component tests added.
- API contract documented and tested.
- Authorization and negative cases tested.
- Database migration/constraints verified.
- UI loading/error/empty states implemented and tested.
- Realtime behavior tested where relevant.
- Observability added for important operational events.
- Security implications reviewed.
- Feature passes staging regression and product acceptance.

28. Final QA Strategy

RoadResQ quality depends on testing the system as a connected operational platform, not merely testing individual screens. The highest-risk areas—identity, object authorization, provider assignment, booking state transitions, realtime location, estimate approval, payments and recovery—must receive the deepest automated and exploratory coverage. Production readiness requires passing functional, security, performance, resilience and operational gates with no unresolved critical defects.